Project: A Smart Order Management System

April 6, 2026

Due Thursday, April 30, 2026 at 11:59PM in Moodle

Collaborators: (LIST OF YOUR COLLABORATORS, OR WRITE NONE)

Team Members: (CAN BE COMPLETED IN TEAMS OF TWO OR THREE.)

Objective:

You are tasked with building a smart and efficient backend system for a bakery. The bakery handles many orders (including same-day and scheduled future orders) and many cancellations per day.

Your task is to design and implement a backend system that stores and manipulates the bakery's order data. You will select appropriate data structures, analyze the efficiency of your operations using Big-O notation, and implement a working system that demonstrates your design.

Goal: The goal is not just functionality, but also design decisions backed by efficient data structures, algorithmic analysis, and careful consideration of trade-offs. *Full credit will be awarded only if your design choices demonstrate efficiency and align well with the system's requirements.*

Data to be Maintained:

Each order will include the following fields:

Field	Type	Description
`order_id`	int or string	Unique identifier for the order
`customer_name`	string	Name of the customer placing the order
`datetime`	DateTime	The exact date and time the order is scheduled to be fulfilled
`status`	string	One of: `"PLACED"`, `"CANCELED"`, `"COMPLETED"`

The following are sample orders:

order_id	customer_name	datetime	status
1001	Noor Hamad	2026-07-25 10:00	COMPLETED
1002	Zain Saleh	2026-07-26 15:00	CANCELED
1003	Jenna Bader	2026-07-25 20:45	PLACED

Required System Operations:

The system must support the following operations, as summarized in the table below:

Operation	Description
`PLACE_ORDER(order)`	Insert a new order, which may be for today or a future date.
`CANCEL_ORDER(order_id)`	Cancel an existing "PLACED" order by order ID.
`PROCESS_NEXT_ORDER()`	Return and remove the next order to be processed. The next order is the one with the earliest scheduled `datetime`.

Your Task:

1. **Data Structures Design.** Choose and justify data structures for efficient operations.
2. **Analyze Algorithm Efficiency.** For each operation, analyze time complexity using Big-O notation with explanations.
3. **Implement the System.** The code should be clean and well-documented.
4. **Communicate Your Design.** In a written report, explain your design decisions, trade-offs, and why your implementation is efficient.

Optional Bonus: Additional Operations:

You may implement **one or two** of the bonus operations listed below. Your implementation must include working code, a clear explanation of your design choices, and a time complexity analysis. *Full credit will be awarded only if your design choices demonstrate efficiency and align well with the system's requirements.*

Custom Operation	Description
`LOYALTY_POINTS()`	Track and retrieve customer loyalty points based on order history.
`FIND_TOP_CUSTOMERS(k)`	Return a list of size k for the most frequent customers.
`VIEW_ALL_ORDERS()`	Display all orders in chronological order.
`VIEW_ALL_ORDERS(order_date)`	Display all orders in chronological order of the given date.

Deliverables:

- **PDF Report:** Complete the provided report template thoroughly.
- **Java Source Code:** Submit all `.java` files. Your code must be clean, well-organized, and include meaningful comments for each method and class.
- **Submission:** Submit all files to **Moodle** by **Thursday, April 30, 2026, at 11:59 PM**. Only one group member should submit on behalf of the team.
- **Demonstration:**
  - You will give a brief live demo of your system (schedule to be announced).
  - Prepare a few slides to guide your explanation.
  - Be ready to explain and justify your design decisions and implementation.

Group Work and Academic Integrity:

- Work must be done in teams of two or three.
- No code sharing or plagiarism allowed.